
Daniel's Cinematic Render Blueprint

A Modular, Deterministic Architecture for High-Fidelity Movie Rendering

Classification: Technical Pipeline Specification | **Version:** 1.0.0 | **Authored:** June 14, 2026

| **Audience:** Daniel — CG Artist / Filmmaker (Intermediate-Advanced) | **Status:**

IMPLEMENTATION READY

This blueprint is a complete, implementation-ready specification for a professional-grade cinematic rendering pipeline. It is not a tutorial — it is an *architecture document*: a rigorous, modular system designed to bring consistency, reproducibility, and cinematic quality to every frame Daniel renders. Each section defines a discrete subsystem that can be adopted independently or integrated as a unified whole. Modular pipeline design matters because cinematic production is inherently iterative: lighting changes, renders rebuild, compositors request new AOVs, and directors revise color intent. Without a locked, modular architecture, these iterations cascade into entropy. With it, every change is scoped, every output is predictable, and every frame is reproducible on demand. Use this document as a living reference — implement the Adapter first, lock Invariants early, and let the Prompt Schema guide AI-assisted lookdev throughout production.

Table of Contents

Section 1 — Render Pipeline Architecture (Modular Design)

- 1.1 Scene Ingestion Layer
- 1.2 Pre-Computation Stage
- 1.3 Primary Ray Pass
- 1.4 Secondary Illumination Pass

- 1.5 Volumetric Integration Pass
- 1.6 Denoising & Reconstruction Stage
- 1.7 Compositing & Post Pass
- 1.8 Output Encoding Stage

Section 2 — Adapter Design Pattern

- 2.1 IRendererAdapter Interface
- 2.2 Renderer Family Notes
- 2.3 Python Pseudocode Stub
- 2.4 Hot-Swap Capability
- 2.5 Parameter Normalization Table

Section 3 — Prompt Schema (AI-Assisted Render Guidance)

- 3.1 RenderPromptSchema Definition
- 3.2 Usage Workflow
- 3.3 Example: Rainy Night City Alley

Section 4 — Motion Curves & Animation Timing

- 4.1 Curve Taxonomy & Bezier Values
- 4.2 Camera Motion Principles
- 4.3 Keyframe Density Guidelines
- 4.4 DCC Integration

Section 5 — Environment Locking Strategy

- 5.1 EnvLock Versioning Schema
- 5.2 Lock Tiers
- 5.3 Lock File Structure
- 5.4 Unlock / Override Protocol
- 5.5 Consistency Checkers

Section 6 — Frame Assembly Logic

- 6.1 Layer Stack Order
- 6.2 AOV Manifest
- 6.3 Comp Node Graph
- 6.4 Alpha Handling Rules
- 6.5 Frame Validation Checklist
- 6.6 Batch Assembly Script

Section 7 — Deterministic Runtime Invariants

- 7.1 Invariant Categories
- 7.2 Invariant Table
- 7.3 Runtime Assertion System
- 7.4 Logging Specification

Section 8 — Implementation Checklist & Getting Started

- 8.1 Quick-Start Checklist
- 8.2 Priority Order
- 8.3 Common Failure Modes
- 8.4 Recommended Folder Structure
- 8.5 Glossary

SECTION 1

Render Pipeline Architecture — Modular Design

The pipeline is divided into eight discrete, ordered stages. Each stage has a defined input contract, a set of operations, and an output contract. No stage may consume data not produced by a prior stage or an external locked asset. This constraint enforces determinism and makes partial re-renders predictable. Stages are designed to be independently restartable — if Stage 5 (Volumetric Integration) fails, you do not rebuild Stages 1-4.

ARCHITECTURAL RULE

Each stage must write its outputs to disk before the next stage begins. In-memory pass-through pipelines are permitted only for preview renders and must be explicitly flagged as non-canonical. Production renders always persist intermediate outputs.

Stage 1

—

Scene Ingestion Layer

Purpose: Load all scene data from source assets, construct the scene graph, and resolve LOD decisions before any computation begins. This is the boundary between asset management and the render runtime.

Inputs: USD/Alembic/FBX scene file(s), asset manifest (JSON), texture atlas references, LOD policy config.

Operations: USD stage composition and layer flattening; scene graph validation (missing references, broken paths); LOD selection based on camera distance heuristics; proxy geometry substitution for non-hero assets; material binding verification; texture UDIM tile enumeration.

Outputs: Resolved scene graph (in-memory or USD flattened layer), asset manifest with resolved paths, LOD assignment log.

Parameter	Default	Range	Notes
lod_distance_threshold_0	0-10 m	1-50 m	Full-detail LOD (hero assets in foreground)
lod_distance_threshold_1	10-50 m	5-200 m	Mid-detail LOD (secondary characters/props)
lod_distance_threshold_2	>50 m	20- ∞ m	Proxy LOD (distant environment, BG dressing)
missing_asset_policy	abort	abort / warn / substitute	Always use abort in production
texture_max_res	4096	256-16384	Per-render-pass override supported
usd_flatten_layers	true	true / false	Must be true for multi-renderer compatibility

Stage 2

—

Pre-Computation Stage

Purpose: Build all acceleration structures and precomputed caches before primary rendering begins. This stage is compute-intensive but fully parallelizable across cores. Its outputs are cached to disk and reused across re-renders unless scene geometry or lighting changes.

Inputs: Resolved scene graph (Stage 1), light source definitions, BVH build config.

Operations: BVH / BLAS+TLAS (ray acceleration structure) construction; light cache warm-up (for biased/hybrid renderers: irradiance cache, photon map prepass, light tree construction); proxy bake for hero assets (displacement to mesh); environment importance map generation from HDRI.

Outputs: Serialized BVH (cache file), light cache (renderer-specific format), precomputed proxy meshes, HDRI importance map.

Parameter	Default	Range	Notes
bvh_build_quality	high	low / medium / high	High quality = slower build, faster trace
light_cache_subdivs	1000	100-8000	V-Ray/hybrid renderers only; ignored by pure PT
cache_invalidate_on_geo_change	true	true / false	Always true in production
hdri_importance_samples	1024	64-8192	Higher = better HDRI sampling, marginal cost
displacement_subdiv_rate	4	1-8	Governs proxy bake resolution
cache_dir	./render_cache/	Any valid path	Must be on fast local SSD, not

Parameter	Default	Range	Notes
			network

Stage 3

—

Primary Ray Pass

Purpose: Dispatch camera rays for every pixel in the image plane, populate the G-buffer with surface attribute data, and collect radiance for direct lighting. This is the foundational pass that all downstream passes depend upon.

Inputs: Resolved scene graph, BVH (Stage 2), camera transform, render resolution, sample count, G-buffer AOV manifest.

Operations: Camera ray generation (pinhole, thin-lens, fisheye per camera model); ray-scene intersection via BVH; G-buffer population per hit point; direct lighting evaluation (shadow rays, MIS); primary ray sample accumulation.

Outputs: G-buffer EXR: **Albedo** (RGB, linear), **World Normals** (XYZ, half-float), **Depth** (Z, 32-bit float), **Motion Vectors** (UV, half-float), **Material ID** (uint16 index), **Direct Diffuse** (RGB), **Direct Specular** (RGB).

Parameter	Default	Range	Notes
primary_samples	512	16-8192	Main quality driver; scale with complexity
camera_model	thin_lens	pinhole / thin_lens / spherical	Use thin_lens for DOF; pinhole for VFX plates
ray_bias	0.0001	0.00001-0.01	Self-intersection offset; tune per scene scale
gbuffer_bit_depth	half (16-bit)	half / full (32-bit)	Use full for depth and motion vector AOVs

Parameter	Default	Range	Notes
<code>motion_vector_shutteropen</code>	-0.25	-0.5-0.0	Relative to frame center; must match Stage 7 DOF
<code>motion_vector_shutterclose</code>	0.25	0.0-0.5	Symmetric shutter by default
<code>adaptive_sampling</code>	true	true / false	Allocates more samples to high-variance regions
<code>adaptive_threshold</code>	0.01	0.001-0.1	Lower = more uniform quality, higher cost

Stage 4

—

Secondary Illumination Pass

Purpose: Compute global illumination by tracing secondary (bounced) rays from G-buffer surface hit points. This stage produces the multi-bounce indirect lighting that gives cinematic renders their depth and realism. It is the most computationally expensive stage.

Inputs: G-buffer (Stage 3), scene BVH (Stage 2), material BSDF definitions, light source definitions, GI config.

Operations: Secondary ray dispatch from G-buffer hit points; recursive path tracing with configurable max depth; Russian Roulette (RR) unbiased path termination; next-event estimation (NEE) for light sampling; MIS weight computation; caustic path handling; indirect diffuse and specular separation.

Outputs: Indirect Diffuse EXR (RGB), Indirect Specular EXR (RGB), SSS (Subsurface Scattering) EXR (RGB), GI raw (full-path combined), path depth statistics log.

⚠ **CRITICAL — Russian Roulette Configuration**

Never disable Russian Roulette in production renders. Without RR, fixed-depth path termination introduces systematic energy loss that cannot be corrected in comp. Set `rr_start_depth` to 3 or higher to let the first bounces complete deterministically before probabilistic termination begins.

Parameter	Default	Range	Notes
<code>max_ray_depth</code>	8	2-32	Total bounce count including primary; 8 is cinema-grade
<code>diffuse_depth</code>	4	1-16	Diffuse bounce sub-limit
<code>specular_depth</code>	4	1-16	Specular/glossy bounce sub-limit
<code>transmission_depth</code>	6	1-16	For glass, liquids, translucent materials
<code>volume_depth</code>	4	1-8	Volume scattering steps within Stage 5
<code>rr_start_depth</code>	3	1-6	Depth at which RR begins; never 0 or 1
<code>rr_threshold</code>	0.1	0.01-0.5	Path survival probability floor
<code>caustics_enabled</code>	<code>false</code>	true / false	Enable only for hero glass/water; costly
<code>gi_clamp_indirect</code>	10.0	1.0-100.0	Indirect firefly suppression; see Stage 6 clamp policy

Stage 5

—

Volumetric Integration Pass

Purpose: Integrate participating media — fog, atmosphere, smoke, clouds, and heterogeneous density fields — along ray paths. Volumetric rendering is physically distinct from surface rendering and requires its own integration pass with dedicated parameters.

Inputs: Scene BVH (Stage 2), volume density fields (VDB or procedural), atmospheric model config, sky model parameters, primary ray paths (Stage 3).

Operations: Raymarching through heterogeneous density fields (OpenVDB integration); single and multiple scattering evaluation; phase function sampling (Henyey-Greenstein); atmospheric Rayleigh and Mie scattering; shadow ray volumetric attenuation; emission from self-illuminating volumes (fire, lava, neon).

Outputs: Volume Color EXR (RGB, premultiplied), Volume Emission EXR (RGB), Atmospheric Scattering AOV (RGB), Volume Depth Mask (single channel).

Parameter	Default	Range	Notes
volume_step_size	0.1	0.01-1.0	Smaller = higher quality, slower; match to VDB voxel size
volume_max_steps	1024	64-8192	Hard limit on raymarching steps
scattering_coefficient	0.1	0.0-10.0	Controls medium density; per-volume override allowed
absorption_coefficient	0.01	0.0-5.0	Energy loss per unit length
phase_g	0.0	-1.0-1.0	Henyey-Greenstein asymmetry; 0=isotropic, >0=forward scatter
atmospheric_model	preetham	preetham / hosek-wilkie / nishita	Nishita = most physically accurate sky

Parameter	Default	Range	Notes
multiple_scattering_depth	2	1-8	Increase for dense clouds; exponential cost increase
volume_shadow_steps	8	4-64	Shadow ray steps through volume per light sample

Stage 6

—

Denoising

&

Reconstruction Stage

Purpose: Apply AI-assisted denoising, suppress fireflies, and optionally accumulate frames temporally to reconstruct a clean, broadcast-ready signal from a statistically noisy path-traced image. This stage bridges raw render output and final compositing.

Inputs: Noisy beauty render (Stage 3+4), G-buffer AOVs (albedo, normals, depth from Stage 3), motion vectors (Stage 3), optional previous-frame denoised output (temporal).

Operations: Pre-denoiser firefly clamping (per-pixel luminance clamp); AI denoiser inference (Intel OIDN v2 or NVIDIA OptiX AI Denoiser); temporal accumulation using reprojected motion vectors; per-AOV denoising (denoise each pass independently before assembly); denoising confidence map generation.

Outputs: Denoised Beauty EXR, per-AOV denoised passes, denoiser confidence map, temporal stability metric log.

❶ **BEST PRACTICE — Denoise Per-AOV, Not Beauty**

Always denoise individual render passes (diffuse, specular, SSS) separately and reassemble in comp rather than denoising the combined beauty. Denoising the combined beauty destroys the frequency separation between passes, making comp adjustments impossible without visible artifacts. The denoised beauty should only be used as a reference check, never as the final deliverable.

Parameter	Default	Range	Notes
denoiser_backend	OIDN	OIDN / OptiX / none	OIDN = CPU/GPU agnostic; OptiX = NVIDIA only but faster
denoiser_prefilter	accurate	none / fast / accurate	Prefilter cleans albedo/normal AOVs before denoising
firefly_clamp_value	20.0	2.0-500.0	Max luminance before clamping; scene-dependent
firefly_clamp_mode	luminance	luminance / per-channel	Luminance mode preserves hue better
temporal_accumulation	false	true / false	Enable for animation; requires locked motion vectors
temporal_blend_factor	0.1	0.05-0.5	Lower = more temporal stability, more ghosting risk
denoise_strength	1.0	0.0-1.0	Blend between raw and denoised; 1.0 = full denoised

Stage 7

—

Compositing

&

Post Pass

Purpose: Apply all in-render color and lens effects that must be computed in linear light space before output encoding. This is distinct from downstream Nuke/DaVinci compositing — this pass handles effects that are physically inseparable from the render (DOF, motion blur bake, lens chromatic aberration, bloom from physical light).

Inputs: Denoised AOV passes (Stage 6), depth AOV (Stage 3), motion vector AOV (Stage 3), camera lens profile, LUT/ACES config.

Operations: Depth-of-field simulation (circle-of-confusion from depth AOV, thin-lens bokeh); motion blur reconstruction from motion vectors; chromatic aberration (lateral CA via per-channel radial warp); physical lens bloom (FFT-based spread from bright sources); vignette (optical falloff); ACES RRT + ODT tone mapping or filmic curve application; 3D LUT application.

Outputs: Tone-mapped beauty EXR (ACES AP1 or scene-linear), lens-effect layers as separate AOVs, graded preview (Rec.709 JPEG for review), post-pass parameter log.

⚠ COLOR SCIENCE NOTE

All in-render tone mapping must be applied *after* all linear-light compositing operations are complete. Never apply a LUT or tone map to individual passes before assembly — doing so destroys linearity and makes correct physical addition of light passes impossible. The render pipeline outputs scene-linear EXR; the ODT (Output Display Transform) is the final operation before delivery encoding.

Parameter	Default	Range	Notes
tone_map_operator	ACES_RRT	ACES_RRT / filmic / Reinhard / none	ACES RRT is industry standard

Parameter	Default	Range	Notes
			for cinema deliverables
aces_exposure_bias	0.0	-4.0-+4.0 EV	Pre-tone-map exposure adjustment in EV stops
dof_bokeh_shape	circular	circular / hexagonal / custom_mask	Custom mask = load aperture PNG for vintage lens feel
dof_fstop	2.8	0.7-22.0	Must match camera invariant (Section 7)
ca_strength	0.002	0.0-0.02	Lateral chromatic aberration magnitude; 0 for clinical look
bloom_threshold	1.0	0.1-10.0	Scene-linear luminance level that triggers bloom spread
bloom_radius	0.05	0.01-0.3	Fraction of image height for bloom kernel radius
vignette_strength	0.3	0.0-1.0	0 = off; 1 = full black at corners
output_colorspace	ACES_AP1	ACES_AP1 / scene_linear / sRGB	EXR deliverable should be ACES_AP1 or scene_linear

Stage 8

—

Output Encoding Stage

Purpose: Encode final render outputs into delivery formats. This stage handles bit-depth decisions, codec selection, color space tagging, and multi-channel EXR

packing. It is the boundary between the render pipeline and the editorial/finishing pipeline.

Inputs: Tone-mapped beauty + all AOV EXRs (Stage 7), output format manifest, delivery specification.

Operations: Multi-channel EXR packing (all AOVs into named channels per OpenEXR spec); codec encode for editorial proxies (ProRes 4444 XQ or DNxHR 444); color space metadata embedding (OCIO config reference); frame naming convention enforcement; checksum generation per frame; delivery manifest write.

Outputs: Multi-channel EXR sequence (archival master), ProRes 4444 or DNxHR 444 proxy (editorial), Rec.709 MP4 review proxy, delivery manifest JSON, per-frame checksum file (SHA-256).

Parameter	Default	Range	Notes
exr_compression	ZIPS	NONE / ZIP / ZIPS / PIZ / DWAB	ZIPS = scanline ZIP; PIZ = best for noisy renders; DWAB = lossy
exr_bit_depth	half	half / full	Use full (32-bit) only for data-critical AOVs (depth, IDs)
proxy_codec	ProRes4444	ProRes4444 / ProRes4444XQ / DNxHR444	Match to editorial NLE; DNxHR for Avid pipelines
frame_naming	[SHOT]_[PASS]_[FRAME%04d]	Custom pattern string	Must be consistent across all AOVs for batch assembly
colorspace_tag	ACES_AP1_Linear	Any OCIO role string	Embedded in EXR metadata for downstream verification
checksum_algorithm	SHA-256	MD5 / SHA-1 / SHA-256	SHA-256 mandatory for archive deliverables
delivery_manifest_format	JSON	JSON / CSV	JSON preferred; machine-readable for automated QC

Adapter Design Pattern

The Adapter Pattern is the architectural keystone that makes this pipeline renderer-agnostic. Instead of hard-coding render commands to a specific renderer (Arnold, Cycles, V-Ray, Redshift, Karma, etc.), the pipeline communicates with a standardized `IRendererAdapter` interface. Each renderer ships a concrete adapter class that translates universal pipeline commands into renderer-specific API calls or command-line arguments. This means Daniel's scene, AOV manifest, and render settings are defined once and execute on any supported renderer without modification.

2.1 IRendererAdapter Interface Specification

The interface defines six required methods. All adapters must implement every method — no optional methods are permitted at the interface level. Renderer-specific extensions are added in the concrete class, not the interface.

Method	Signature	Responsibility	Must Raise On Failure
<code>initialize()</code>	<code>initialize(config: dict) -> bool</code>	Connect to renderer runtime, validate license, set working directory, load OCIO config	<code>RendererInitError</code>
<code>loadScene()</code>	<code>loadScene(scene_path: str, manifest: dict) -> SceneHandle</code>	Load scene file into renderer context, validate asset resolution, return opaque scene handle	<code>SceneLoadError</code>
<code>setRenderSettings()</code>	<code>setRenderSettings(settings: RenderSettings) -> None</code>	Apply all normalized render parameters from the universal settings object to renderer internals	<code>SettingsValidationError</code>

Method	Signature	Responsibility	Must Raise On Failure
<code>executePass()</code>	<code>executePass(pass_id: str, frame: int) -> PassResult</code>	Execute a named render pass for the given frame; block until completion; return <code>PassResult</code> with status and output paths	<code>RenderPassError</code>
<code>fetchAOV()</code>	<code>fetchAOV(aov_name: str, frame: int) -> AOVBuffer</code>	Retrieve a named AOV buffer by name and frame number; normalize channel layout to universal format	<code>AOVNotFoundError</code>
<code>teardown()</code>	<code>teardown() -> None</code>	Release renderer resources, flush logs, close IPC connections, write session summary	<code>TeardownError</code> (logged, not propagated)

2.2 Renderer Family Implementation Notes

Renderer Family	Examples	Adapter Strategy	Key Differences
Unbiased Path Tracers	Cycles, Karma (HDPR), Mitsuba	Direct parameter mapping; no cache pre-pass; purely sample-based quality control	No light cache; RR is native; samples = primary quality lever
Biased / Hybrid	V-Ray, Arnold (adaptive), Redshift	Must call Stage 2 light cache pre-pass via adapter; maps samples to "min/max samples" pair	Light cache warm-up required; biased approximations need explicit disable for hero shots
GPU-Accelerated	Redshift, Octane, VRAY-GPU	Adapter must manage VRAM budget; texture streaming config; device enumeration	VRAM overflow = silent quality degradation; must validate before production run

Renderer Family	Examples	Adapter Strategy	Key Differences
Spectral Renderers	Corona (spectral mode), Mitsuba 3	Color inputs must be spectral SPDs, not RGB; adapter converts scene RGB to D65 SPD on load	Fluorescence, dispersion, metamerism require spectral path; ACES output mapping differs

2.3 Python Pseudocode Stub

```
# ===== #
IRendererAdapter – Base Interface (Python Pseudocode) # Daniel's Cinematic
Render Blueprint – Section 2 #
===== from abc import
ABC, abstractmethod from dataclasses import dataclass from typing import Any
@dataclass class RenderSettings:      primary_samples: int      # Universal
sample count      max_ray_depth: int      # Total bounce limit
clamp_value: float      # Firefly clamp threshold      resolution: tuple
# (width, height) in pixels      colorspace: str      # OCIO colorspace
string      aov_manifest: list      # List of AOV name strings to output
frame_range: tuple      # (start_frame, end_frame)      output_dir: str
# Absolute path for render output      random_seed: int      #
Deterministic seed (Section 7) @dataclass class PassResult:      pass_id: str
frame: int      status: str      # "success" | "failed" | "aborted"
output_paths: dict      # {"aov_name": "/path/to/file.exr"}
render_time_seconds: float      sample_count: int @dataclass class AOVBuffer:
aov_name: str      frame: int      width: int      height: int      channels:
list      # ["R", "G", "B"] or ["X", "Y", "Z"] etc.      bit_depth: str
# "half" | "full"      data_ref: Any      # Opaque reference to buffer
data class IRendererAdapter(ABC):      """      Universal renderer interface.
All concrete adapters must implement      all six methods without exception.
No optional methods permitted.      """      @abstractmethod      def
initialize(self, config: dict) -> bool:      """      Connect to
renderer runtime. Validate license. Load OCIO config.      Set working
directory. Return True on success.      Raise RendererInitError on any
failure.      """      raise NotImplementedError      @abstractmethod
def loadScene(self, scene_path: str, manifest: dict):      """
Load scene into renderer. Validate all asset references.      Apply LOD
policy from manifest. Return opaque SceneHandle.      Raise SceneLoadError
if any required asset is missing.      """      raise
NotImplementedError      @abstractmethod      def setRenderSettings(self,
settings: RenderSettings) -> None:      """      Apply normalized
RenderSettings to renderer internals.      Translate universal params to
renderer-specific equivalents.      Validate all values against renderer
limits before applying.      Raise SettingsValidationError on out-of-range
or unsupported values.      """      raise NotImplementedError
@abstractmethod      def executePass(self, pass_id: str, frame: int) ->
PassResult:      """      Execute named render pass for given frame.
Block until complete.      Write outputs per Stage 8 naming convention.
Return PassResult with status, output paths, render time.      Raise
RenderPassError on render failure; include renderer log path.      """
raise NotImplementedError      @abstractmethod      def fetchAOV(self,
aov_name: str, frame: int) -> AOVBuffer:      """      Retrieve AOV
buffer by name and frame. Normalize channel layout.      Validate bit
```

```

depth against AOV manifest spec.          Raise AOVNotFoundError if AOV was
not rendered or cannot be found.          """         raise
NotImplementedError @abstractmethod def teardown(self) -> None:
"""         Release all renderer resources. Flush logs.         Write session
summary to render log.         Catch and log TeardownError – do not re-raise.
"""         raise NotImplementedError # ----- Concrete Example: Cycles
Adapter (stub) ----- class CyclesAdapter(IRendererAdapter): def
initialize(self, config: dict) -> bool:         # Import bpy (Blender Python
API)         # Set render engine to CYCLES         # Apply OCIO config path
# Validate GPU/CPU device selection         return True # Placeholder
def setRenderSettings(self, settings: RenderSettings) -> None:         #
bpy.context.scene.cycles.samples = settings.primary_samples         #
bpy.context.scene.cycles.max_bounces = settings.max_ray_depth         #
bpy.context.scene.cycles.sample_clamp_direct = settings.clamp_value         #
bpy.context.scene.cycles.seed = settings.random_seed         pass # -----
Concrete Example: Arnold Adapter (stub) ----- class
ArnoldAdapter(IRendererAdapter): def initialize(self, config: dict) ->
bool:         # import arnold as ai         # ai.AiBegin()         #
ai.AiMsgSetConsoleFlags(ai.AI_LOG_ALL)         # Load Arnold license via
ARNOLD_LICENSE_HOST env var         return True         def
setRenderSettings(self, settings: RenderSettings) -> None:         #
ai.AiNodeSetInt(options, "AA_samples", settings.primary_samples)         #
ai.AiNodeSetInt(options, "GI_total_depth", settings.max_ray_depth)         #
ai.AiNodeSetFlt(options, "AA_sample_clamp", settings.clamp_value)         #
ai.AiNodeSetInt(options, "AA_seed", settings.random_seed)         pass

```

2.4 Hot-Swap Capability

Hot-swapping the renderer mid-project — changing from Arnold to Redshift partway through a production — is safe under the adapter pattern provided the following conditions are met:

- **Scene is authored in USD:** The scene must be renderer-neutral (USD). No renderer-specific node types or shaders may be in the base scene layer. Renderer-specific shading is provided via USD variant sets that are resolved at load time by each adapter's `loadScene()` call.
- **RenderSettings is universal:** The `RenderSettings` object is populated from the pipeline config file, not from any renderer UI. This ensures the same quality settings apply to both renderers.
- **AOV manifest is renderer-validated:** Before executing a pass with a new renderer, the adapter's `setRenderSettings()` must validate that all requested AOVs in the manifest are supported. Unsupported AOVs raise a warning and are skipped — they do not abort the render.

- **Invariants are re-locked:** After a renderer swap, all Section 7 invariants must be re-validated. The pre-render assertion system (Section 7.3) handles this automatically on session start.

2.5 Parameter Normalization Table

Universal Parameter	Cycles (Blender)	Arnold	V-Ray	Redshift	Karma (Houdini)
Primary Samples	<code>cycles.samples</code>	<code>AA_samples</code>	<code>subdivs</code> (<code>subdivs²=samples</code>)	<code>UnifiedMaxSamples</code>	<code>karma:global:samplesperpixel</code>
Max Ray Depth	<code>cycles.max_bounces</code>	<code>GI_total_depth</code>	<code>maxDepth</code>	<code>RS_max_ray_depth</code>	<code>karma:maxraydepth</code>
Diffuse Depth	<code>cycles.diffuse_bounces</code>	<code>GI_diffuse_depth</code>	<code>diffuseDepth</code>	<code>RS_diffuse_depth</code>	<code>karma:diffusedepth</code>
Specular Depth	<code>cycles.glossy_bounces</code>	<code>GI_specular_depth</code>	<code>reflectionDepth</code>	<code>RS_specular_depth</code>	<code>karma:reflectiondepth</code>
Clamp Value	<code>sample_clamp_direct</code>	<code>AA_sample_clamp</code>	<code>clampOutput</code>	<code>RS_clampValue</code>	<code>karma:sampleclamping</code>
Random Seed	<code>cycles.seed</code>	<code>AA_seed</code>	<code>dmcs_randomSeed</code>	<code>RS_random_seed</code>	<code>karma:seed</code>
Motion Blur Shutter	<code>render.motion_blur_shutter</code>	<code>shutter_start/end</code>	<code>moblur_intervalCenter</code>	<code>RS_motionblur_shutterOpen/Close</code>	<code>karma:shutter</code>

SECTION 3

Prompt Schema — AI-Assisted Render Guidance

The `RenderPromptSchema` is a structured data specification that Daniel fills out to describe a shot's look intent in machine-readable form. It serves two roles: (1) a communication document for the production — everyone on the team reads the same schema to understand the shot's visual goal; and (2) a direct input to AI tools

(large language models, AI lookdev assistants, or AI-driven material generators) that translate the schema into concrete lighting setups, material parameters, and color grade directions.

3.1 RenderPromptSchema Definition

The schema is authored in YAML for human readability. A JSON equivalent is generated automatically by the pipeline tooling and used for machine interfaces.

```
# ===== #
RenderPromptSchema v1.0 # Daniel's Cinematic Render Blueprint – Section 3 #
Usage: Fill this schema per shot. Pass to AI lookdev # assistant or attach to
shot brief in production tracker. #
===== schema_version:
"1.0" shot_id: "SHOT_ID" # e.g., "SC005_SH0030" project_id:
"PROJECT_ID" # ----- Scene Intent ----- scene_intent: mood: "string"
# e.g., "melancholic", "tense", "ethereal" genre: "string"
# e.g., "neo-noir", "sci-fi thriller", "period drama" time_of_day: "string"
# e.g., "blue hour dusk", "noon harsh sunlight" weather: "string"
# e.g., "heavy rain", "overcast", "clear night" narrative_beat: "string"
# e.g., "revelation moment", "chase", "quiet introspection" # ----- Primary
Light Source ----- primary_light_source: type: "string"
# sun | area | point | spot | HDRI | practical color_temp_K: 0
# Kelvin; 2700=tungsten, 5600=daylight, 8000=overcast sky intensity_lux:
0.0 # Physical lux value at subject; 100000=noon sun
direction_vector: [0.0, 0.0, 0.0] # World-space XYZ direction (normalized)
angular_diameter_deg: 0.0 # For sun: 0.53; for area: derived from
size/distance shadow_softness: "string" # hard | soft | very_soft
# ----- Material Descriptors ----- material_descriptors: - asset_name:
"string" # e.g., "hero_jacket", "wet_asphalt" surface_type:
"string" # diffuse | metal | glass | SSS | emission | mixed
roughness_range: [0.0, 1.0] # [min, max] roughness; use midpoint as base
value metalness: 0.0 # 0.0=dielectric, 1.0=fully
metallic IOR: 1.5 # Index of refraction;
1.5=glass, 1.33=water, 2.4=diamond subsurface_radius_mm: [0,0,0] #
[R,G,B] SSS radius in mm; [8,3,2] typical skin notes: "string"
# e.g., "rain-wet surface, puddle reflections active" # ----- Atmosphere
----- atmosphere: fog_density: 0.0 # 0.0=clear, 1.0=heavy
fog; maps to scattering_coefficient scattering_coefficient: 0.0 #
Direct volume param override (Section 1, Stage 5) dust_particles: false
# Enable fine particle mote simulation rain_streak_density: 0.0 #
0.0-1.0; drives particle system density atmospheric_model: "preetham" #
preetham | hosek-wilkie | nishita # ----- Camera ----- camera:
focal_length_mm: 35 # Physical focal length; 24=wide,
50=normal, 85=portrait aperture_fstop: 2.8 # Physical
aperture; lower=shallower DOF shutter_angle: 180 # Degrees;
180=standard cinema (1/48s at 24fps) sensor_size: "Super35" #
Super35 | FullFrame | IMAX | custom [W,H mm] focus_distance_m: 0.0
# Distance to focus plane in world units camera_motion: "string"
# static | handheld | dolly | crane | drone | rig # ----- Color Grade Intent
----- color_grade_intent: "string" # Free-form description of final
grade target # e.g., "desaturated teal-
orange noir with crushed blacks" # e.g.,
"warm golden-hour Terrence Malick soft glow"
# e.g., "cold clinical blue-white clinical fluorescent" # ----- Reference
Films ----- reference_films: - "string" # e.g.,
```

```

"Blade Runner 2049 – night exteriors" - "string" #
e.g., "Arrival – grey overcast diffuse lighting" - "string"
# e.g., "Dune Part Two – harsh desert backlight" # ----- AI Instruction
Override ----- ai_instruction: "string" # Optional: direct
instruction for AI lookdev assistant #
e.g., "Prioritize specular wetness on hero surface.
# Use key-to-fill ratio of 8:1. Motivate all
# secondary lights from practical neon signs."

```

3.2 Usage Workflow

1. **Fill the schema at shot brief stage** — before modeling or shading begins.
The schema defines the visual target and prevents lookdev from diverging from the director's intent.
2. **Pass scene_intent + primary_light_source + color_grade_intent + reference_films to an AI lookdev assistant** (e.g., a GPT-4o API call, a specialized CG AI tool, or a Nuke Copilot plugin). Request concrete: HDRI selection, key light placement angles, fill ratio, rim light presence.
3. **Pass material_descriptors to an AI material generator** — request specific roughness curves, normal map frequency guidance, and layered shader construction notes per asset.
4. **Feed camera parameters directly into the pipeline's RenderSettings object** — these values are invariants (Section 7) and must not change between lookdev and final render.
5. **Archive the filled schema alongside the shot's EnvLock file** (Section 5) — it becomes part of the shot's permanent record for re-render reproducibility.

3.3 Example Filled Schema — Rainy Night City Alley

```

# ===== #
RenderPromptSchema v1.0 – FILLED EXAMPLE # Shot: SC012_SH0045 | "Alley
Confrontation" # Project: NOCTURNE_2026 #
===== schema_version:
"1.0" shot_id: "SC012_SH0045" project_id: "NOCTURNE_2026" scene_intent:
mood: "oppressive, dread, isolation" genre: "neo-noir crime thriller"
time_of_day: "2:00 AM – deep night, no ambient sky light" weather: "heavy
rain, standing puddles, wet surfaces throughout" narrative_beat:
"protagonist discovers a body – quiet horror, no action"
primary_light_source: type: "practical_motivated_area" color_temp_K: 2900
# Sodium vapor streetlamp – warm amber intensity_lux: 180 #

```

```

Dim, underlit alley – streetlamp is 10m above    direction_vector: [0.0, -1.0,
0.1] # Straight down with slight forward tilt    angular_diameter_deg: 0.0
# Area light – no angular diameter    shadow_softness: "soft" # Large
source at distance creates soft penumbra    material_descriptors: -
asset_name: "wet_asphalt"    surface_type: "mixed"    roughness_range:
[0.02, 0.15] # Nearly mirror-wet; puddle areas near 0.02    metalness: 0.0
IOR: 1.33    # Water surface IOR    subsurface_radius_mm:
[0,0,0]    notes: "Use layered shader: dry asphalt base + rain-wet top layer
blended    by procedural puddle mask. Puddles should reflect neon
signs directly." - asset_name: "hero_jacket"    surface_type: "mixed"
roughness_range: [0.4, 0.7] # Wet leather – lower roughness than dry
metalness: 0.0    IOR: 1.52    subsurface_radius_mm: [0,0,0]    notes:
"Black leather jacket, rain-soaked. Specular highlight from
streetlamp should read as a single sharp-edged bright streak." -
asset_name: "skin_hero"    surface_type: "SSS"    roughness_range: [0.35,
0.5]    metalness: 0.0    IOR: 1.4    subsurface_radius_mm: [8, 3, 2] #
Standard Caucasian skin approximation    notes: "Face partially in shadow.
Subsurface should be visible only on    ear rim and jawline where
streetlamp backlit. No rim light – silhouette."    atmosphere:    fog_density:
0.08    # Light atmospheric haze, not heavy fog
scattering_coefficient: 0.05    dust_particles: false    rain_streak_density:
0.7    # Heavy rain streaks visible in light cones    atmospheric_model:
"hosek-wilkie" # Hosek-Wilkie handles artificial light better    camera:
focal_length_mm: 75    # Slightly telephoto – compressed space,
paranoia    aperture_fstop: 1.8    # Wide aperture – shallow DOF,
background bokeh    shutter_angle: 180    # Standard cinema 1/48s at
24fps    sensor_size: "Super35"    focus_distance_m: 2.2    #
Protagonist face at 2.2m – body falls OOF    camera_motion: "handheld"    #
Micro-shake – barely perceptible, adds unease    color_grade_intent:
"Desaturated teal-blue shadows with deep crushed blacks.    Sodium amber for
midtones motivated by streetlamp. Highlights land at a    cool off-white – no
warm rolloff in highlights. Heavy vignette.    Reference: Se7en (1995) rain
sequences, True Detective S1 night exteriors."    reference_films: - "Se7en
(1995) – night rain sequences, Andrew Lesnie / Darius Khondji" - "True
Detective Season 1 – bayou night exteriors, Adam Arkapaw" - "Blade Runner
2049 – artificial light motivated scenes, Roger Deakins" - "Prisoners
(2013) – desaturated, overcast, oppressive palette, Roger Deakins"
ai_instruction: "Generate a 3-point lighting breakdown for this setup.
Motivate    all secondary fill from off-screen neon signage (specify color
temps and angles).    Provide specular roughness values for wet asphalt that
will show visible neon    reflections. Suggest a V-Ray or Arnold shader
network for the puddle layer blend.    Color grade: target S-Log3/Venice as
capture format, grade to Rec.709 delivery."

```

SECTION 4

Motion Curves & Animation Timing

Cinematic motion is defined not by where things go but by how they get there. The difference between amateur and professional animation is almost entirely in the curve — the acceleration profile that governs how a value changes over time. Linear interpolation reads as mechanical; cinematic motion is characterized by weighted

eases, micro-overshoot, and secondary settling. This section defines Daniel's standard curve library and the principles for applying it correctly.

4.1 Curve Taxonomy & Bezier Control Point Values

All curves are defined as cubic Bezier curves with four control points: P0 (start), P1 (first handle), P2 (second handle), P3 (end). Values are normalized to the range [0,1] for both time (X) and value (Y). Scale to actual timing and value ranges in the DCC.

Curve Name	P0	P1 (Out Handle)	P2 (In Handle)	P3	Primary Use Case
Ease-In-Out (Cubic)	(0.0, 0.0)	(0.42, 0.0)	(0.58, 1.0)	(1.0, 1.0)	General camera moves, object translation, standard animation
Ease-In-Out (Soft)	(0.0, 0.0)	(0.25, 0.0)	(0.75, 1.0)	(1.0, 1.0)	Floating objects, aerial shots, dream sequences
Anticipation Settle	(0.0, 0.0)	(0.3, -0.1)	(0.6, 1.08)	(1.0, 1.0)	Character head turns, prop pickups, door swings with slight overshoot
Cinematic Drift	(0.0, 0.0)	(0.05, 0.0)	(0.95, 1.0)	(1.0, 1.0)	Ultra-slow crane moves, parallax drift, tidal/water simulation
Inertia Snap	(0.0, 0.0)	(0.0, 0.8)	(0.7, 1.05)	(1.0, 1.0)	Quick head snaps, fast cuts with hold, subtitle or UI pop-on

Curve Name	P0	P1 (Out Handle)	P2 (In Handle)	P3	Primary Use Case
Whip Pan Ease	(0.0, 0.0)	(0.1, 0.0)	(0.4, 1.0)	(1.0, 1.0)	Whip pan transitions, action scene camera swings
Heavy Settle	(0.0, 0.0)	(0.5, 0.0)	(0.8, 1.04)	(1.0, 1.0)	Heavy machinery, vehicle stops, earthquake-style impacts
Elastic Overshoot	(0.0, 0.0)	(0.6, 0.0)	(0.7, 1.15)	(1.0, 1.0)	Stylized animation, cartoon physics, spring-like elements

⚠ **NEVER USE LINEAR INTERPOLATION IN PRODUCTION**

Linear interpolation ($P1 = P0 + (P3 - P0) * 0.333$, $P2 = P0 + (P3 - P0) * 0.666$) produces mechanical, robotic motion that is immediately visible to audiences. The only valid use of linear curves is for technical data channels (shader parameter ramps, color correction values) that are explicitly intended to be mechanically precise. All spatial animation — position, rotation, scale — must use a shaped curve from the library above or a production-approved custom curve.

4.2 Camera Motion Principles

- **The Weighted Tangent Rule:** Camera keyframes must always use weighted (non-normalized) tangent handles in the DCC. Normalized tangents scale handle length with key spacing, which destroys the easing relationship when keys are moved in time. Weighted tangents preserve the absolute handle length and must be set explicitly for every camera key.

- **Lag/Lead Offset for Secondary Motion:** Camera secondary motion (shake, micro-jitter, lens breathing) must be offset by 2–4 frames behind the primary move. Primary move leads; secondary shake follows. A simultaneous start for primary and secondary motion reads as artificial. In Maya, use an Animation Layer with a +2 frame offset for the shake layer. In Blender NLA, use the strip time offset parameter.
- **No Camera Holds in Mid-Move:** If a camera move must pause between two positions, the hold must be created by extending the ease-out of the first move and ease-in of the next move until they share a flat tangent region — never by a discontinuous stop followed by a restart. A true velocity-zero hold is acceptable only at the start and end of a move, not in the middle.
- **Rotation Before Translation:** When a camera both rotates and translates, rotation changes should lead translation by 3–8 frames. The camera looks where it's going before it goes there — this matches the natural human vestibular response and reads as intentional camera operation rather than mechanical rigging.

4.3 Keyframe Density Guidelines

Animation Type	Strategy	Key Density	Curve Shape Rule
Hero character pose-to-pose	Sparse keys + curve shaping	3–8 keys per second max	Each key gets individual tangent attention; no auto-tangent
Facial animation (performance capture)	Dense baked + curve reduce	24–120 keys/sec raw, reduce to 6–12 after filter	Apply Butterworth filter at 8Hz before delivery
Camera translation (dolly/crane)	Sparse keys + curve shaping	2–4 keys per move	Always Cinematic Drift or Ease-In-Out (Soft); never Auto
Camera rotation (pan/tilt)	Sparse keys + curve shaping	2–3 keys per move	Rotation leads translation by 3–8 frames (see 4.2)

Animation Type	Strategy	Key Density	Curve Shape Rule
Cloth / secondary dynamics	Baked simulation	24 keys/sec (every frame)	No curve editing — baked output only; adjust in simulation
Rigid body / vehicle	Baked simulation or sparse FK	Baked: 24/sec; FK: 4-6/sec	Apply heavy-settle curve to stops; linear on constant velocity segments
Shader / material animation	Sparse keys	2 keys per transition	Ease-In-Out acceptable; linear valid for technical values

4.4 DCC Integration — Applying Curves

DCC	Tool Location	Key Actions
Maya — Graph Editor	Windows > Animation Editors > Graph Editor	Select keys > Tangents > Weighted; set tangent type to Custom; paste handle XY values from table above in absolute world space. Use Tangent Speed input field for precise handle lengths.
Blender — NLA & Curve Editor	Editor Type: Graph Editor; NLA Editor for layer offset	Set handle type to "Aligned" (not "Auto"). For weighted tangents, use handle type "Free" and manually set handle positions. Apply the Simplify operator (F-Curve > Clean) after bake, threshold 0.001.
Houdini — CHOP Network	Network editor > Insert CHOP	Use the Spring CHOP for secondary lag/overshoot simulation. Use the Lag CHOP for lead/follow offset. Wire camera animation through a Filter CHOP (Butterworth, 8Hz cutoff) before output. Export via Channel WRANGLER to object

DCC	Tool Location	Key Actions
		transforms.
Nuke — Curve Editor	Curve Editor panel (for animated knob values)	Paste Bezier control points as TCL expressions. Use the "Smooth" tangent mode and manually adjust handles. For camera shake: use the ScanlineRender camera rig with Expression-linked noise on translate channels.

SECTION 5

Environment Locking Strategy

Environment consistency across multi-shot productions is one of the most common sources of continuity errors in CG work. If Shot 12 and Shot 14 both take place in the same location but were rendered two weeks apart, subtle differences in HDRI rotation, sun azimuth, or fog density create a "wrongness" that audiences feel even if they cannot articulate it. The EnvLock system eliminates this problem by treating the lighting environment as a versioned, locked artifact with the same discipline applied to geometry and animation caches.

5.1 EnvLock Versioning Schema

Every environment configuration is identified by a canonical lock ID:

ENVLOCK-[ProjectID]-[SceneID]-[Version]-[Timestamp] Examples: ENVLOCK-NOCTURNE2026-ALLEY01-v003-20260614T1235 ENVLOCK-NOCTURNE2026-R00FTOP-v001-20260601T0900 ENVLOCK-NOCTURNE2026-ALLEY01-v004-20260615T0815 (post-review revision)
Fields: ProjectID : Alphanumeric, no spaces, max 16 chars (e.g., NOCTURNE2026) SceneID : Alphanumeric scene identifier, max 16 chars (e.g., ALLEY01) Version : v[NNN] zero-padded 3-digit version (e.g., v003) Timestamp : ISO 8601 compact, minute precision (e.g., 20260614T1235)

5.2 Lock Tiers

Tier	Name	State	Who Can Modify	Downstream Impact
0	Draft	Active development; parameters change freely	Any lookdev artist	No downstream shots locked to this tier
1	Soft Lock	Under review by supervisor/director; no modifications without approval note	Lookdev lead + approval note required	Test renders permitted; no production renders
2	Hard Lock	Director-approved; no modifications except via formal override protocol (5.4)	VFX Supervisor only; requires change log entry	All shots in scene render against this env version
3	Archive Lock	Final delivery complete; file is read-only; no overrides permitted	No one; file system permissions set to read-only	Retained for re-delivery and archival; no active pipeline connection

5.3 Lock File Structure

```
// EnvLock File – JSON Format // Filename: ENVLOCK-NOCTURNE2026-ALLEY01-v003-20260614T1235.json {  "lock_id": "ENVLOCK-NOCTURNE2026-ALLEY01-v003-20260614T1235",  "lock_tier": 2,  "lock_tier_name": "HARD_LOCK",  "locked_by": "daniel_td",  "locked_at": "2026-06-14T12:35:00-04:00",  "approved_by": "sarah_vfxsup",  "approved_at": "2026-06-14T12:30:00-04:00",  "project_id": "NOCTURNE2026",  "scene_id": "ALLEY01",  "version": "v003",  "environment": {    "hdri": {      "file": "hdri/nocturne_alley_2am_overcast_v003.hdr",      "sha256": "a3f4c9d2e1b5a7f8c3d4e2f1a0b9c8d7e6f5a4b3c2d1e0f9a8b7c6d5e4f3a2",      "rotation_y_deg": 147.5,      "intensity_multiplier": 0.08,      "colorspace": "linear_rec709"    },    "sun": {      "enabled": false,      "azimuth_deg": 0.0,      "elevation_deg": 0.0,      "intensity_lux": 0.0    },    "sky_model": {      "model": "hosek-wilkie",      "turbidity": 6.5,      "albedo": 0.1,      "sun_disk_visible": false    },    "ground_plane_shader": {      "file": "shaders/wet_asphalt_hero_v007.mtlx",      "sha256": "b9c8d7e6f5a4b3c2d1e0f9a8b7c6d5e4f3a2b1c0d9e8f7a6b5c4d3e2f1a0b9",      "puddle_mask_file": "masks/alley01_puddle_mask_v002.exr",      "puddle_mask_sha256": "c1d2e3f4a5b6c7d8e9f0a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2"    },    "fog_settings": {      "enabled": true,      "density": 0.08,      "scattering_coefficient": 0.05,      "absorption_coefficient": 0.008,      "color_linear_rgb": [0.12, 0.14, 0.18],      "height_falloff": 0.3,    }  }
```

```

"rain_streak_density": 0.7      },      "time_of_day_metadata":
{      "scene_time": "02:00",      "season": "autumn",
"latitude_deg": 40.7,      "longitude_deg": -73.9,      "timezone":
"America/New_York"      },      "practical_lights":
[      {      "light_id": "streetlamp_01",      "type": "area",
"color_temp_K": 2900,      "intensity_lux": 180.0,      "position_xyz":
[0.0, 10.2, -3.5],      "direction_xyz": [0.0, -1.0, 0.1],      "file":
"lights/streetlamp_IES_sodium_v002.ies",      "ies_sha256":
"d2e3f4a5b6c7d8e9f0a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3"      }
]      },      "changelog": [      {      "version": "v001",      "date": "2026-
06-01T09:00:00-04:00",      "author": "daniel_td",      "change": "Initial
draft environment setup"      },      {      "version": "v002",      "date":
"2026-06-10T14:22:00-04:00",      "author": "daniel_td",      "change":
"Adjusted HDRI rotation +12 deg; reduced fog density 0.12->0.08 per director
note"      },      {      "version": "v003",      "date": "2026-06-
14T12:35:00-04:00",      "author": "daniel_td",      "change": "Added
sodium streetlamp IES; updated wet asphalt shader v007; director approved"
}      ] }

```

5.4 Unlock / Override Protocol

A Hard Lock (Tier 2) or Archive Lock (Tier 3) may only be overridden through the following formal protocol. No informal overrides are permitted — all overrides are permanently recorded in the lock file changelog.

6. **Override Request:** The requesting artist submits an override request describing the specific parameter to change, the reason, and the shots affected. This is logged in the project tracker with priority level.
7. **VFX Supervisor Approval:** Only the VFX Supervisor (or delegated TD Lead) may approve a Hard Lock override. No artist may override their own Hard Lock. Director approval is required if the change affects any director-approved shot.
8. **Version Increment:** The override always creates a new version (v003 → v004). The old version is never modified in place. Both versions remain on disk.
9. **Changelog Entry Required:** The new version's changelog must include: author, approver, timestamp, changed parameter(s), old values, new values, reason, and list of affected shots.
10. **Shot Re-render Trigger:** The override system automatically flags all shots referencing the old EnvLock ID for re-render review. The artist must confirm

which shots need re-render vs. which can absorb the change without re-rendering.

11. **Archive Lock is immutable:** Archive Lock (Tier 3) files may not be overridden under any circumstances. If an archived environment must be modified for re-delivery, a new scene ID suffix is created (e.g., ALLEY01_REDELIVER_2027).

5.5 Consistency Checkers — Automated Parameter Diff Tool

The pipeline includes an automated env-diff check that runs before every production render. It compares the active scene's environment parameters against the locked EnvLock file and flags any deviation above threshold.

```
# ===== # EnvLock
Consistency Checker — Pseudocode # Runs automatically at the start of every
production render. # Aborts render if any CRITICAL deviation is detected. #
===== def
check_env_consistency(scene_env: dict, lock_file_path: str,
abort_on_critical: bool = True) -> CheckResult:    lock =
load_json(lock_file_path)    deviations = []    # --- HDRI Checks ---
if scene_env["hdri"]["rotation_y_deg"] != lock["environment"]["hdri"]
["rotation_y_deg"]:    delta = abs(scene_env["hdri"]["rotation_y_deg"]
- lock["environment"]["hdri"]["rotation_y_deg"])    severity =
"CRITICAL" if delta > 0.5 else "WARNING"    deviations.append({"param":
"hdri.rotation_y_deg",    "locked":
lock["environment"]["hdri"]["rotation_y_deg"],
"active": scene_env["hdri"]["rotation_y_deg"],
"delta": delta,    "severity": severity})    if
scene_env["hdri"]["sha256"] != lock["environment"]["hdri"]["sha256"]:
deviations.append({"param": "hdri.file_hash",
"locked": lock["environment"]["hdri"]["sha256"],
"active": scene_env["hdri"]["sha256"],    "delta":
"HASH_MISMATCH",    "severity": "CRITICAL"})    #
--- Fog Checks ---    fog_delta = abs(scene_env["fog"]["density"]
- lock["environment"]["fog_settings"]["density"])    if fog_delta > 0.005:
deviations.append({"param": "fog.density",
"locked": lock["environment"]["fog_settings"]["density"],
"active": scene_env["fog"]["density"],    "delta":
fog_delta,    "severity": "WARNING" if fog_delta <
0.02 else "CRITICAL"})    # --- IES Profile Hash Check ---    for light in
lock["environment"]["practical_lights"]:    scene_light =
find_scene_light(scene_env, light["light_id"])    if
scene_light["ies_sha256"] != light["ies_sha256"]:
deviations.append({"param": f"light.{light['light_id']}.ies_hash",
"locked": light["ies_sha256"],    "active":
scene_light["ies_sha256"],    "delta":
"HASH_MISMATCH",    "severity": "CRITICAL"})
critical_count = sum(1 for d in deviations if d["severity"] == "CRITICAL")
write_deviation_report(deviations, lock["lock_id"])    if abort_on_critical
```

```

and critical_count > 0:
    raise
    EnvLockViolationError(
        f"Render ABORTED: {critical_count}
        CRITICAL env deviations "
        f"vs lock {lock['lock_id']}. See
        deviation report."
    )
    return CheckResult(deviations=deviations,
        critical_count=critical_count,
        lock_id=lock["lock_id"])

```

SECTION 6

Frame Assembly Logic

Frame assembly is the process of taking raw render passes (AOVs) from the render pipeline and combining them into a final deliverable frame. Correct assembly requires precise knowledge of which operation (add, multiply, screen, over) combines each pass, what alpha convention is expected, and in what order layers are stacked. Incorrect assembly is one of the most common sources of silent errors in CG production — images that look approximately right but are subtly wrong in ways that compound across hundreds of frames.

6.1 Layer Stack Order — Bottom to Top

Stack Position	Layer Name	Blend Operation	Alpha Expected	Notes
1 (Bottom)	Background Plate	Base layer (no blend)	None (solid)	Live action plate or procedural BG environment
2	Diffuse GI (Indirect)	Add over background	Premultiplied	Indirect diffuse from Stage 4; must be denoised separately
3	Direct Diffuse	Add	Premultiplied	Direct diffuse from Stage 3; add to Diffuse GI for full diffuse
4	Specular (Direct + Indirect)	Add	Premultiplied	Combined specular; keep separate from diffuse for

Stack Position	Layer Name	Blend Operation	Alpha Expected	Notes
				comp flexibility
5	SSS (Subsurface Scattering)	Add	Premultiplied	Add only for SSS materials (skin, wax, vegetation)
6	Volume / Atmosphere	Over (premultiplied)	Premultiplied	Volume emission adds; atmospheric scatter uses Over with Volume Depth Mask
7	FX Passes	Over / Add (per effect)	Premultiplied	Particle FX, lens flares, sparks — blend mode varies by effect
8	Matte Painting / Extensions	Over	Premultiplied	Digital matte painting extensions; occlusion from hero elements composited on top
9 (Top)	Color Grade / LUT	Applied globally to merged result	N/A	ACES RRT + creative LUT applied to full-assembled linear frame before ODT

6.2 AOV Manifest Table

AOV Name	Format	Bit Depth	Merge Operation	Purpose
beauty	RGB EXR	Half	Reference only	Combined render preview; never used as

AOV Name	Format	Bit Depth	Merge Operation	Purpose
				final comp input
diffuse_direct	RGB EXR	Half	Add	Direct illumination on diffuse surfaces
diffuse_indirect	RGB EXR	Half	Add	GI bounced light on diffuse surfaces
specular_direct	RGB EXR	Half	Add	Direct specular highlights
specular_indirect	RGB EXR	Half	Add	Indirect specular (reflections of environment)
sss	RGB EXR	Half	Add	Subsurface scattering contribution
emission	RGB EXR	Half	Add	Self-illuminating surfaces (screens, fire, LEDs)
volume_scatter	RGB EXR	Half	Over (premult)	Atmospheric / participating media scatter
volume_emission	RGB EXR	Half	Add	Self-illuminating volumes (fire, lava)
albedo	RGB EXR	Half	Denoiser input only	Base color without lighting; used by OIDN/OptiX denoiser
world_normals	XYZ EXR	Half	Denoiser input only	World-space normals; used by denoiser and relighting tools
depth_z	Float EXR	Full (32-bit)	DOF / distance	World-space

AOV Name	Format	Bit Depth	Merge Operation	Purpose
			matte	depth in scene units; used for DOF and fog effects
motion_vectors	UV EXR	Half	Motion blur / temporal	Per-pixel screen-space motion; used for MB reconstruction and denoiser temporal
material_id	UInt16 EXR	Full (16-bit uint)	Selection mask generation	Per-pixel material index; used to generate per-material selection mattes
object_id	UInt16 EXR	Full (16-bit uint)	Selection mask generation	Per-pixel object index; used to isolate individual objects in comp
shadow_catcher	RGBA EXR	Half	Multiply	Shadow contribution on ground/contact areas; multiply over background plate
reflection_catcher	RGBA EXR	Half	Screen / Add	Reflection contribution; used for reflection augmentation in comp
cryptomatte	RGBA EXR x3	Full	Matte extraction tool	Cryptomatte rank 0-2; enables ID-based matte generation in Nuke/Fusion

6.3 Comp Node Graph Description

The following describes the canonical comp network in Nuke (adapt node equivalents for Fusion or DaVinci Resolve Fusion):

12. **Read nodes** — one per AOV, reading EXR sequence with correct frame padding. Set `colorspace = ACES_AP1_Linear` on all linear AOVs.
13. **Merge (Plus) — Diffuse Full:** Connect `diffuse_direct` (A) and `diffuse_indirect` (B) into a Merge node, operation: Plus. Output: `diffuse_full`.
14. **Merge (Plus) — Specular Full:** Connect `specular_direct` + `specular_indirect` into Merge Plus. Output: `specular_full`.
15. **Merge (Plus) — Surface:** Add `diffuse_full` + `specular_full` + `sss` + `emission`. Output: `surface_combined`.
16. **Merge (Over) — Volume:** Merge `volume_scatter` Over `surface_combined` using Volume Depth Mask as alpha. Then Plus-merge `volume_emission`.
17. **Merge (Over) — FX:** Over-merge each FX layer in order of depth (far to near, use `depth_z` to resolve order). Each FX element must carry its own alpha.
18. **Merge (Over) — Matte Painting:** Over-merge matte painting extensions; apply held-out hero element alpha to punch back through the extension.
19. **ACES Transform node:** Apply ACES RRT using an OCIOColorSpace node (transform from `ACES_AP1_Linear` to `Output_Rec709`). This is the final tone-mapping step.
20. **Grade / LUT:** Apply creative LUT using a Vectorfield node or OCIOFileTransform. Grade node for global and shadow/midtone/highlight adjustments.
21. **Write node:** Output to ProRes 4444 (review) and as DPX or TIFF sequence (edit delivery).

6.4 Alpha Handling Rules

Pass Type	Alpha Convention	Rule
All additive lighting passes (diffuse, specular, SSS, emission)	No alpha channel required	These passes are always added, never composited Over. Alpha is irrelevant and should be discarded on read.
Volume / Atmosphere passes	Premultiplied	Volume alpha encodes density. Always use premultiplied; never unpremultiply before Merge Over operations.
FX / Particle passes	Premultiplied	Particle systems must render with premultiplied alpha. Straight-alpha particle renders are a pipeline error.
Matte painting extensions	Straight alpha (for artist layers)	Photoshop/Procreate matte painting layers export as straight alpha. Premultiply before entering comp (Nuke: Premult node immediately after Read).
Hero CG elements over live plate	Premultiplied	All renderer-generated alpha is premultiplied. Verify with a Shuffle node: premultiplied black areas are black; straight black areas have colour.
Cryptomatte channels	N/A	Cryptomatte channels are not composited; they are used by the Cryptomatte plugin as ID data only. Never connect to Merge operations.

6.5 Frame Validation Checklist

Run this checklist against every frame before it is approved for editorial delivery.

Automated QC scripts should cover items 1-10; items 11-15 require human review.

22. **Black Frame Detection:** Mean luminance of beauty render is above minimum threshold (default: 0.001 in scene-linear). Zero-mean frames = render failure.
23. **Highlight Clipping Check:** No pixel in the beauty render exceeds `max_scene_linear_value` (default: 100.0). Values above indicate uncontrolled fireflies that escaped the clumper.
24. **Alpha Range Validation:** Alpha channel values are in [0.0, 1.0] range. Out-of-range alpha = shader or compositing error.
25. **Motion Vector Sanity:** Max motion vector magnitude does not exceed `frame_width` × 0.5 in pixels. Larger values indicate non-converged motion data or a degenerate camera jump.
26. **Depth Range Validation:** Depth Z AOV minimum value is greater than zero (no negative depth). Maximum depth does not exceed `camera_clip_far`. Infinite depth (camera ray miss) encoded as exactly `camera_clip_far`, not NaN or Inf.
27. **NaN/Inf Pixel Detection:** No NaN or Inf values in any AOV. A single NaN propagates through additive comp operations and corrupts all downstream frames.
28. **Naming Convention Compliance:** Filename matches the pattern `[SHOT]_[PASS]_[FRAME%04d].exr`. Mismatched names break automated frame assembly.
29. **Frame Number Sequence Continuity:** No missing frame numbers in sequence. Gaps in sequence = dropped frames; assembly script must abort, not skip.
30. **AOV Channel Count Verification:** Each AOV EXR contains the expected number of channels as defined in the AOV Manifest (6.2). Extra or missing channels indicate a render settings drift.
31. **Colorspace Metadata Tag:** EXR metadata `chromaticities` and `colorspace` fields match the expected `ACES_AP1_Linear` tag. Untagged EXRs must be rejected.

32. **SHA-256 Checksum Verification:** Per-frame checksum matches the value recorded in the delivery manifest. Failed checksum = file corruption during transfer.
33. **EnvLock ID Match:** Render log contains EnvLock ID that matches the current approved Hard Lock for the scene. Mismatched ID = environment drift; re-render required.
34. **Denoiser Confidence Map Review:** Any region with denoiser confidence below 0.3 is flagged for human review. Low confidence = insufficient samples; may need sample boost.
35. **Motion Blur Coherence:** Motion blur direction on hero moving objects is visually coherent with motion vector AOV. Inconsistent direction = motion vector or shutter timing error.
36. **Human QC — Continuity Check:** Frame is visually compared against adjacent frames (± 5) for lighting continuity, flicker, shadow popping, or edge aliasing. Automated tools cannot fully replace human continuity review.

6.6 Batch Assembly Script — Pseudocode

```
# ===== # Batch Frame
Assembly – Pseudocode # Reads AOV manifest, validates all passes, assembles #
comp tree, and writes delivery frames automatically. # Daniel's Cinematic
Render Blueprint – Section 6 #
===== import json from
pathlib import Path def batch_assemble(manifest_path: str, frame_range:
tuple, output_dir: str, abort_on_error: bool = True):
manifest = load_json(manifest_path) shot_id = manifest["shot_id"]
aovs = manifest["aov_list"] # List of AOV dicts from Section 6.2
envlock_id = manifest["envlock_id"] for frame in range(frame_range[0],
frame_range[1] + 1): print(f"[ASSEMBLY] Assembling frame {frame:04d}
for {shot_id}") # --- Step 1: Validate all AOV files exist ---
missing = [] for aov in aovs: expected_path =
build_path(output_dir, shot_id, aov["name"], frame) if not
Path(expected_path).exists(): missing.append(expected_path)
if missing: handle_error(f"Missing AOVs for frame {frame}:
{missing}", abort=abort_on_error)
continue # --- Step 2: Validate checksums --- for aov in
aovs: aov_path = build_path(output_dir, shot_id, aov["name"],
frame) recorded_sha = manifest["checksums"]
[f"{aov['name']}_{frame:04d}"] actual_sha = sha256(aov_path)
if actual_sha != recorded_sha: handle_error(f"Checksum FAIL:
{aov_path}", abort=abort_on_error) continue # ---
Step 3: Run pre-assembly frame validation --- validation_result =
validate_frame(shot_id, frame, aovs, envlock_id) if
```

```

validation_result.critical_failures > 0:                handle_error(f"Frame
{frame} failed QC: "
f"{validation_result.critical_failures} critical issues",
abort=abort_on_error)                                continue          # --- Step 4: Load AOV
buffers ---          buffers = {}                    for aov in aovs:
aov_path = build_path(output_dir, shot_id, aov["name"], frame)
buffers[aov["name"]] = load_exr(aov_path)              # --- Step 5: Assemble
layer stack (Section 6.1 order) ---                  surface =
(              buffers["diffuse_direct"]              +
buffers["diffuse_indirect"]                          + buffers["specular_direct"]
+ buffers["specular_indirect"]                        + buffers["sss"]          +
buffers["emission"]                                  )              volume =
merge_over(buffers["volume_scatter"], surface,
alpha=buffers["volume_scatter"].alpha)                volume = volume +
buffers["volume_emission"]                          fx_merged = merge_fx_layers(buffers,
depth_aov=buffers["depth_z"])                        assembled = merge_over(fx_merged,
volume)          # --- Step 6: Apply ACES RRT (scene-linear -> display) ---
graded = apply_ocio_transform(assembled,
src="ACES_AP1_Linear",
dst="Output_Rec709")                                # --- Step 7: Write delivery frame ---
out_path = build_delivery_path(output_dir, shot_id, frame)
write_exr(graded, out_path, compression="ZIPS", bit_depth="half")          #
--- Step 8: Write review proxy ---                    proxy_path =
build_proxy_path(output_dir, shot_id, frame)          write_prores4444(graded,
proxy_path)          print(f"[ASSEMBLY] Frame {frame:04d} complete ->
{out_path}")          print(f"[ASSEMBLY] All frames complete for {shot_id}.")

```

SECTION 7

Deterministic Runtime Invariants

Determinism in rendering means that given the same inputs, a render always produces bit-identical (or within floating-point tolerance) output. In a production environment, this means: if a render needs to be re-run two weeks later — because of a delivery error, a hard drive failure, or a client revision — it will produce exactly the same result. Without locked invariants, this guarantee is impossible. With them, it is absolute.

⚠ INVARIANTS ARE NOT DEFAULTS

An invariant is not a parameter with a sensible default that can be overridden. It is a value that is locked for the duration of a render session and may only be changed by explicitly incrementing the session version and invalidating the associated cache. Any tool that treats an invariant as an overridable render

setting is a pipeline error.

7.1 Invariant Categories

1. Seed Invariants

Every stochastic sampling operation — primary ray sampling, light selection, BSDF sampling, volume step jitter — must use a deterministic seed that is a pure function of `(frame_number, pass_id, render_session_id)`. No renderer may use wall-clock time, process ID, or any other non-deterministic value as a seed component. The seed function is: `seed = hash(frame_number || pass_id || session_random_base_seed)` where `session_random_base_seed` is set once at session start and locked.

2. Geometry Invariants

Subdivision level, displacement amplitude, and tessellation rate must be frozen to their approved values before any render begins. Dynamic subdivision (where subdivision level varies per frame based on camera distance) is prohibited in production renders — it introduces frame-to-frame geometry change that makes temporal denoising and motion vector consistency impossible.

3. Shading Invariants

Texture filter mode (bilinear, trilinear, anisotropic level), MIP bias, and BRDF model version (GGX vs. Beckmann, exact version of the Cook-Torrance implementation) must be locked. BRDF model version is particularly critical: many renderers ship updates between point releases that subtly change the energy conservation behavior of BSDFs, producing different reflectance values for identical roughness inputs.

4. Lighting Invariants

IES profile files must be locked by SHA-256 hash (as implemented in the EnvLock system, Section 5). Physical sky model version (which atmospheric model and which version of it) must be logged. Any change to an IES profile — even correcting a data entry error — must be treated as a new version that invalidates all affected renders.

5. Camera Invariants

The projection matrix (derived from focal length, sensor size, resolution, and pixel aspect ratio) is computed once at session start and locked. The sensor response curve version (for physically-based camera exposure simulation) is locked. Any change to camera optics — even a focal length adjustment of 0.1mm — must be treated as a new camera version requiring full re-render.

6. Temporal Invariants

Frame rate, shutter timing (open/close relative to frame center), and motion blur sample count must be fixed for the entire sequence. Mixed frame rates within a single comp tree are prohibited. If a sequence requires a frame rate change (e.g., a slow-motion insert), it must be treated as a separate sub-sequence with its own invariant set.

7.2 Invariant Table

Invariant	Value Type	Lock Method	Validation Check
Random Base Seed	uint64	Set in session config JSON; read-only after session start	Assert seed value matches session config on every pass start
Frame Rate	rational (e.g., 24000/1001)	Embedded in USD layer; read at session start; locked	Assert against USD layer on render call; abort on mismatch
Shutter Open / Close	float pair (relative to frame center)	RenderSettings object; locked after <code>setRenderSettings()</code>	Assert before every frame dispatch; log mismatch
Motion Blur Samples	int	RenderSettings object; locked after <code>setRenderSettings()</code>	Assert before every frame; abort on mismatch
Subdivision Level (per object)	int per object ID	USD variant set; frozen on scene load (Stage 1)	Assert subdivision level matches locked value at render time
Displacement Amplitude	float per material ID	MaterialX shader lock; SHA-256	Verify .mtlx SHA-256 against lock

Invariant	Value Type	Lock Method	Validation Check
		hash of .mtlx file	at session start
Texture Filter Mode	enum (bilinear/trilinear/aniso)	Renderer config; set once at initialize(); read-only	Query renderer filter mode after init; assert matches config
Texture MIP Bias	float	Renderer config; locked	Assert MIP bias value at session start
BRDF Model + Version	string + version tag	Renderer version pinned in Docker/environment config	Log renderer version string; fail if not matching pinned version
Focal Length (mm)	float	Camera USD prim; locked at scene load	Assert camera focal length matches RenderPromptSchema camera.focal_length_mm
Sensor Size	float pair [W, H mm]	Camera USD prim; locked at scene load	Assert against scene load; abort if mismatch vs. session config
Aperture f-stop	float	Camera USD prim; locked at scene load	Cross-check against Stage 7 DOF parameters; must be identical
IES Profile (per light)	SHA-256 hash	EnvLock file (Section 5); verified at session start	Hash on-disk IES file; compare against EnvLock; abort on mismatch
Sky Model + Version	string + version tag	EnvLock file + renderer version pin	Assert sky model string matches EnvLock; log in render log
HDRI File Hash	SHA-256 hash	EnvLock file (Section 5)	Hash HDRI file on disk; compare against EnvLock; abort on mismatch
Output Resolution	int pair [W, H px]	Session config; locked after setRenderSettings()	Assert all AOVs match session resolution; abort if any differ

7.3 Runtime Assertion System

```
# ===== # Pre-Render
Assertion System – Pseudocode # Called automatically before every render
pass. # Any ABORT-level failure terminates the render immediately. # WARNING-
level failures are logged but do not abort. #
===== class
RenderAssertionSystem: def __init__(self, session_config: dict,
lock_file_path: str): self.session_config = session_config
self.lock = load_json(lock_file_path) self.failures = [] def
assert_invariant(self, name: str, expected, actual,
tolerance: float = 0.0, severity: str = "ABORT"): if tolerance > 0.0:
if abs(float(expected) - float(actual)) > tolerance:
self.failures.append({ "invariant": name,
"expected": expected, "actual": actual,
"severity": severity, "delta": abs(float(expected) -
float(actual)) }) else: if expected !=
actual:
self.failures.append({ "invariant": name,
"expected": expected, "actual": actual,
"severity": severity, "delta": "EXACT_MISMATCH"
}) def run_all_checks(self, renderer_adapter, scene_env: dict) -> bool:
cfg = self.session_config lock_env = self.lock["environment"]
# --- Seed Invariants --- self.assert_invariant("random_seed",
cfg["random_seed"],
renderer_adapter.get_seed(), severity="ABORT") # --- Camera
Invariants --- self.assert_invariant("focal_length_mm",
cfg["camera"]["focal_length_mm"],
scene_env["camera"]["focal_length_mm"],
tolerance=0.001, severity="ABORT")
self.assert_invariant("aperture_fstop",
cfg["camera"]["aperture_fstop"],
scene_env["camera"]["aperture_fstop"],
tolerance=0.001, severity="ABORT")
self.assert_invariant("shutter_open",
cfg["shutter_open"],
renderer_adapter.get_shutter_open(),
tolerance=0.0001, severity="ABORT") # --- Temporal Invariants ---
self.assert_invariant("frame_rate",
cfg["frame_rate"], scene_env["frame_rate"],
severity="ABORT") self.assert_invariant("motion_blur_samples",
cfg["motion_blur_samples"],
renderer_adapter.get_mb_samples(), severity="ABORT") # ---
Environment Invariants (via EnvLock) ---
self.assert_invariant("hdri_sha256",
lock_env["hdri"]["sha256"],
sha256(scene_env["hdri"]["file"]), severity="ABORT") for
locked_light in lock_env["practical_lights"]: active_light =
find_scene_light(scene_env, locked_light["light_id"])
self.assert_invariant(f"ies_sha256_{locked_light['light_id']}",
locked_light["ies_sha256"],
sha256(active_light["ies_file"]), severity="ABORT") # --- Shading
Invariants --- self.assert_invariant("renderer_version",
cfg["pinned_renderer_version"],
renderer_adapter.get_version(), severity="ABORT") # --- Evaluate
Results --- abort_failures = [f for f in self.failures if
f["severity"] == "ABORT"] warn_failures = [f for f in self.failures
if f["severity"] == "WARNING"] write_assertion_log(self.failures,
cfg["render_log_path"]) if abort_failures: raise
```

```
InvariantViolationError(
    {len(abort_failures)} ABORT-level "
    Render terminated. See log: "
)
    return len(warn_failures) == 0 # True = clean, False = warnings
present
    f"Pre-render assertion FAILED:
    f'invariant violations.
    f'{cfg['render_log_path']}'}"
```

7.4 Logging Specification

Every render session must produce a structured render log that enables full post-mortem reproducibility. The log is append-only (never overwritten) and stored alongside the render output in the `render_logs/` directory.

Log Entry Type	Required Fields	Purpose
SESSION_START	timestamp, session_id, renderer_version, renderer_name, python_version, os_platform, envlock_id, all invariant values	Complete snapshot of session state at start; enables full reproduction
ASSERTION_RESULT	timestamp, invariant_name, expected, actual, severity, pass/fail	Pre-render assertion results; fail entries trigger render abort
PASS_START	timestamp, pass_id, frame, sample_target, random_seed_for_frame	Records per-frame seed for exact reproduction of any single frame
PASS_COMPLETE	timestamp, pass_id, frame, actual_samples, render_time_s, output_paths, sha256_per_output	Confirms outputs written; checksums for integrity verification
ENV_DIFF_RESULT	timestamp, envlock_id, deviation_count, critical_count, full deviation list	Records any environment parameter drift detected before render
ERROR	timestamp, error_type, error_message, stack_trace, frame, pass_id	Full error context; enables diagnosis without re-running render
SESSION_COMPLETE	timestamp, total_frames, total_time_s, failed_frames list, warnings list, envlock_id	Summary for production tracking; failed frames list for re-render queue

Implementation Checklist & Getting Started

8.1 Quick-Start Checklist

Use this checklist when plugging the blueprint into an existing project workflow for the first time:

Step	Task	Section Ref	Estimated Effort
1	Author a concrete <code>IRendererAdapter</code> for your primary renderer	Section 2	1-2 days
2	Define <code>RenderSettings</code> config JSON for your project defaults	Section 2.5	2-4 hours
3	Lock all Section 7 invariants in session config; pin renderer version	Section 7	4-8 hours
4	Create initial <code>EnvLock</code> files (Draft tier) for all environments	Section 5	1 day per environment
5	Fill <code>RenderPromptSchema</code> for all hero shots	Section 3	30-60 min per shot
6	Configure Stage 3 (Primary Ray) AOV manifest for your renderer	Section 1.3	4-8 hours
7	Set up denoiser (OIDN or OptiX) and validate per-	Section 1.6	4-8 hours

Step	Task	Section Ref	Estimated Effort
	AOV output		
8	Build comp template per Section 6.3; validate with a single test frame	Section 6	1–2 days
9	Run frame validation checklist (Section 6.5) on first assembled frame	Section 6.5	2–4 hours
10	Import curve library into DCC Graph Editor; apply to first camera move	Section 4	2–4 hours
11	Promote EnvLock files from Draft to Soft Lock; schedule director review	Section 5.2	Project-dependent
12	Run first full production render; review render log for invariant warnings	Section 7.4	Full render duration

8.2 Priority Implementation Order

① RECOMMENDED IMPLEMENTATION ORDER

Do not try to implement the entire blueprint at once. This order minimizes rework and ensures the highest-risk items (renderer coupling, environment drift, reproducibility) are addressed before lower-risk items (curve library, prompt schema).

37. **Adapter** (Section 2) — Decouples you from any single renderer immediately. Every hour spent building the Adapter saves days later when a renderer must be swapped.
38. **Invariants** (Section 7) — Lock the seed, the camera, and the renderer version before the first production render. Retroactively applying invariants to existing renders is nearly impossible.
39. **Environment Lock** (Section 5) — Lock environments as soon as they receive director approval. Even a Draft lock is better than no lock.
40. **Pipeline Architecture** (Section 1) — Implement Stages 1-4 for the primary ray and GI pass. Stages 5-8 can be integrated progressively as shots require them.
41. **Prompt Schema** (Section 3) — Fill schemas for hero shots during lookdev; integrate AI tools at whichever point they add value to the workflow.
42. **Frame Assembly** (Section 6) — Build the comp template and validation checklist before the first round of hero renders are delivered.
43. **Curve Library** (Section 4) — Import into DCC and apply to camera animation as shots enter animation. Not urgent until animation begins.

8.3 Common Failure Modes and How to Avoid Them

Failure Mode	Symptom	Root Cause	Prevention
Flickering between frames	Luminance or color pops on consecutive frames	Non-deterministic seed; temporal denoiser ghosting; subdivision level change between frames	Lock random seed (Invariant 1); enable temporal accumulation in Stage 6; freeze subdivision (Invariant 2)
Lighting mismatch between shots	Same scene location looks different in shots rendered weeks apart	No EnvLock or EnvLock not enforced	Implement Section 5; enforce consistency checker before every production render

Failure Mode	Symptom	Root Cause	Prevention
Fireflies in final comp	Bright single-pixel spikes visible in motion	Clamp value too high or disabled; denoiser not applied	Set <code>firefly_clamp_value</code> (Stage 6); always denoise per-AOV
Premultiplication errors in comp	Dark halo around composited CG elements	Mixing premultiplied and straight alpha operations	Enforce alpha convention table (Section 6.4); add Premult node immediately after every straight-alpha Read
Re-render produces different result	Client requests re-render; output differs from original delivery	Invariants not locked; renderer version not pinned; seed not deterministic	Implement all Section 7 invariants; pin renderer in Docker/virtualenv; log all invariant values in <code>SESSION_START</code>
Adapter swap breaks scene	Switching from Arnold to Redshift changes material appearance	Scene contains renderer-specific shader nodes in base USD layer	Keep base USD layer renderer-neutral; all renderer-specific shading in USD variant sets
Denoising destroys fine detail	Fabric weave, pore detail, or text blur in denoised output	Denoising beauty combined rather than per-AOV; denoiser strength too high	Denoise per-AOV only (Stage 6 best practice); tune <code>denoise_strength</code> per material type; use confidence map to identify over-smoothed regions
Volume pass incorrectly blended	Fog appears to occlude objects it should not; double-fog in some areas	Volume merged as Add instead of Over; incorrect Volume Depth Mask usage	Strictly follow Section 6.1 layer stack; Volume pass is always Over, never Add

8.4 Recommended Folder / Project Structure

```

/[PROJECT_ID]/ /pipeline/      config.json          # Session
config: invariants, renderer version, defaults      adapter/
IRendererAdapter.py          # Base adapter interface      CyclesAdapter.py

```



```

ArnoldAdapter.py          RedshiftAdapter.py          schemas/
RenderPromptSchema_v1.yaml # Schema template (Section 3)          tools/
env_checker.py            # EnvLock consistency checker (Section 5.5)
frame_validator.py        # Frame validation checklist (Section 6.5)
batch_assembler.py        # Batch frame assembly (Section 6.6)
assertion_system.py       # Invariant assertion system (Section 7.3)
/assets/                  /geo/                  # USD scene geometry
layers                    /shaders/              # MaterialX .mtlx shader files
(lock by SHA-256)         /textures/             # UDIM texture sets
/hdri/                   # HDRI lighting files (referenced in EnvLock)
/ies/                    # IES light profiles (referenced in EnvLock)
/vdb/                    # OpenVDB volume caches          /envlocks/
ENVLOCK-[PROJ]-[SCENE]-v001-[TIMESTAMP].json  ENVLOCK-[PROJ]-[SCENE]-v002-
[TIMESTAMP].json          /shots/          /[SCENE_ID]/          /[SHOT_ID]/
prompt_schema.yaml        # Filled RenderPromptSchema (Section 3)
render_settings.json      # Shot-specific render settings overrides
aov_manifest.json         # Shot AOV list (Section 6.2)          /renders/
/[PASS_NAME]/             [SHOT]_[PASS]_0001.exr
[SHOT]_[PASS]_0002.exr    /assembly/          [SHOT]_assembled_0001.exr
[SHOT]_assembled_0001_proxy.mov    /delivery_manifests/
[SHOT]_delivery_manifest.json      [SHOT]_checksums.sha256
/render_cache/            /bvh/                  # Serialized BVH
acceleration structures (Stage 2)    /light_cache/          # Light
cache files per renderer (Stage 2)    /render_logs/
[SESSION_ID]_[TIMESTAMP].log # Append-only render session logs (Section
7.4)    /comp/          template_master.nk          # Nuke comp template
(Section 6.3)    [SHOT_ID].nk          # Per-shot Nuke scripts

```

8.5 Glossary

Term	Definition
AOV	Arbitrary Output Variable. A named render pass output channel (e.g., direct diffuse, normals, depth). AOVs are assembled in comp to produce the final image.
ACES	Academy Color Encoding System. The industry-standard color management framework for film production. AP0 is the scene-referred colorspace; AP1 (ACEScg) is used for rendering and compositing; the RRT + ODT converts to display.
BVH	Bounding Volume Hierarchy. A tree data structure that spatially partitions scene geometry to accelerate ray-scene intersection tests. Construction happens once in Stage 2; traversal happens for every ray.
BRDF	Bidirectional Reflectance Distribution Function. The mathematical model

Term	Definition
	governing how light reflects from a surface. Common models: GGX, Beckmann, Oren-Nayar (diffuse). Version consistency is a shading invariant.
BSDF	Bidirectional Scattering Distribution Function. The superset of BRDF that includes transmission (BTDF). Used for glass, translucent materials, and subsurface scattering.
Caustics	Focused light patterns caused by reflective or refractive surfaces (e.g., light through a glass of water). Computationally expensive to resolve with pure path tracing; typically requires photon mapping or dedicated caustic algorithms.
Cryptomatte	An open standard for generating per-object/per-material ID mattes automatically during rendering. Stored as multiple RGBA EXR layers; decoded in comp (Nuke, Fusion) to generate clean selection mattes without manual roto.
DOF	Depth of Field. The range of distances in a scene that appear acceptably sharp. Controlled by aperture f-stop, focal length, and focus distance. Simulated via thin-lens model in Stage 7.
EXR	OpenEXR. The standard high-dynamic-range image format for VFX production. Supports multiple named channels, floating-point bit depths (half/full), and lossless/lossy compression. All render pipeline intermediates use EXR.
EnvLock	The versioned, checksummed environment configuration file defined in Section 5. Contains all lighting environment parameters necessary to reproduce a rendered frame's environment exactly.
Firefly	A bright, isolated pixel spike in a path-traced render caused by a high-variance, low-probability light path being sampled. Suppressed by

Term	Definition
	luminance clamping in Stage 6.
G-Buffer	Geometry Buffer. A collection of texture-like AOVs (albedo, normals, depth, motion vectors, material ID) that encode surface properties at primary ray hit points. Used by the denoiser and as inputs to secondary passes.
GI	Global Illumination. Lighting that accounts for indirect light bouncing between surfaces, producing color bleeding, soft ambient illumination, and the characteristic richness of physically-based renders.
HDRI	High Dynamic Range Image. A 360° panoramic image capturing a real lighting environment, used as an image-based light (IBL) source. Provides realistic ambient lighting and reflections.
IES Profile	Illuminating Engineering Society photometric data file. Describes the angular distribution of light emitted by a physical luminaire. Used to simulate real-world light fixtures with physical accuracy.
Invariant	In this pipeline, a parameter that is locked for the duration of a render session and may not change without explicitly incrementing the session version. Invariants are the foundation of deterministic rendering. See Section 7.
LOD	Level of Detail. A technique that substitutes lower-complexity geometry representations for objects based on their distance from the camera, reducing render cost without visible quality loss.
MaterialX	An open standard for representing material and look definitions in a renderer-agnostic format. Used in this pipeline as the canonical format for all surface shader definitions, locked by SHA-256.
MIS	Multiple Importance Sampling. A variance reduction technique that

Term	Definition
	combines sampling strategies from different distribution functions (BSDF and light) to reduce noise without bias.
OCIO	OpenColorIO. The open-source color management library used across the VFX industry. Manages all color space transforms in the pipeline via a config file that defines all roles and transforms.
OIDN	Intel Open Image Denoise. An open-source, hardware-agnostic AI denoiser that uses trained neural networks to remove Monte Carlo noise from path-traced renders. Supported as a primary denoiser backend in Stage 6.
ODT	Output Display Transform. In ACES terminology, the final transform that converts scene-referred (AP1 linear) image data to a display-referred output suitable for a specific display device (e.g., Rec.709 monitor, P3 DCI projector).
Path Tracing	A physically-based rendering algorithm that simulates light transport by stochastically tracing ray paths from the camera through the scene. Unbiased in the limit; converges to ground truth with sufficient samples.
Russian Roulette	An unbiased path termination strategy that stochastically ends low-contribution ray paths, compensating for the termination by boosting surviving paths' contribution. Prevents both infinite recursion and biased fixed-depth cutoff.
SSS	Subsurface Scattering. Light that penetrates below a surface and scatters multiple times before exiting. Essential for realistic rendering of skin, marble, wax, milk, and vegetation.
USD	Universal Scene Description. Pixar's open standard for 3D scene composition and interchange. The canonical scene format for this pipeline, enabling renderer-agnostic scene authoring and efficient layer-based

Term	Definition
	composition.
VDB	OpenVDB. The industry-standard sparse volumetric data format, used for storing smoke, fire, clouds, and other heterogeneous density fields. Stage 5 (Volumetric Integration) operates directly on VDB grids.
VRAM	Video RAM. GPU memory. GPU-accelerated renderers (Redshift, Octane, V-Ray GPU) must fit the entire scene into VRAM. Overflow causes silent degradation (texture streaming, geometry paging) that must be explicitly managed by the adapter.

Daniel's Cinematic Render Blueprint — Version 1.0.0 — June 14, 2026

This document is a living technical specification. As the pipeline evolves, increment the version number, update the changelog in your session config, and re-validate all invariants against the new version. Every structural change to the pipeline must be treated as a potential re-render trigger for in-flight shots.